

Graph-Based Intelligent Movie Recommendation System Using RAG

Project Report

Presented To

Prof. Kaikai Liu

In Partial Fulfillment of the Requirements for the Course

CMPE 258: Deep Learning – Spring 2026

Department of Software Engineering

San José State University

Team ID: Group 19

Project Title: Graph-Based Intelligent Movie Recommendation System Using RAG

Project Track: System / Application

Focused Areas: Knowledge Graphs, Retrieval-Augmented Generation (RAG), Recommender Systems, Semantic Search, FAISS, Large Language Models, Evaluation-First AI Systems

Team Members

Name	SJSU ID	Email
Bharath Kumar A	018221268	bharathkumar.a@sjsu.edu
Saripella Sriyavarma	019130553	sriyavarma.saripella@sjsu.edu
Venkata Siva Sai Krishna Prasad Yedupati	018320835	venkatasivasaikrishnaprasad.yedupati@sjsu.edu

Code Repository:

<https://github.com/abharathkumarr/Graph-Based-Intelligent-Movie-Recommendation-System-Using-RAG>

May 2026

Contents

1	Abstract	2
2	Introduction and Problem Description	2
3	Background and Related Work	3
4	Dataset	4
4.1	Node Types	4
4.2	Relationship Types	4
4.3	Dataset Visualizations	5
5	System / Model / Algorithm Design	6
6	Implementation Details	7
6.1	Languages and Frameworks	7
6.2	Compute Environment	7
6.3	Key Implementation Decisions	7
6.4	Pipeline Implementation Steps	8
7	Task Distribution and Contributions	10
8	Evaluation and Testing Results	11
8.1	Metrics	11
8.2	Focused Qualitative Validation	11
8.3	Extended Evaluation: 51 Queries x 3 Models	11
8.4	Model Quality and Latency Comparison	12
8.5	Precision and Recall vs. Retrieval k	12
8.6	Per-Query Retrieval Metrics	13
8.7	Retrieval Confusion Matrix	13
8.8	Findings	13
8.9	Reproduction Steps	14
9	User Interface and Demo	14
10	Conclusion and Future Work	15

1. Abstract

Traditional movie recommendation systems based on collaborative or content-based filtering struggle with cold-start, popularity bias, and lack of interpretability.

We present a **Graph-Based Intelligent Movie Recommendation System** that combines a Neo4j knowledge graph with a Retrieval-Augmented Generation (RAG) pipeline.

We extract 229,894 subject-predicate-object triplets from the Neo4j *recommendations* dataset, render them as natural-language sentences, embed them with SentenceTransformers (`all-MiniLM-L6-v2`), and index them in FAISS for sub-second semantic retrieval.

A LangChain RetrievalQA chain feeds the top- k retrieved triplets to a Groq-hosted LLM, which produces fact-grounded answers with citations.

We deliver an evaluation-first study on 51 natural-language queries across three Groq LLMs, with 15 Cypher-derived ground-truth queries, and report Precision@10, Recall@10, F1@10, latency, and rate-limit/context-window failure modes.

The evaluation drives an explicit production choice (`llama-3.1-8b-instant`: F1@10 = 0.32 at 3.0s mean latency) and exposes real-world deployment trade-offs that pure offline accuracy comparisons hide.

2. Introduction and Problem Description

Movie recommendation is a high-impact application of machine learning, but classical solutions are dominated by collaborative filtering and content-based approaches with well-known weaknesses: popularity bias, cold-start for new users or items, and lack of explainability.

As streaming catalogs grow into the hundreds of thousands of titles, users increasingly issue natural-language queries such as “*a recent adventure film with Leonardo DiCaprio rated above 7*” that simple matrix-factorization models cannot answer.

Our project addresses this with a hybrid **Knowledge-Graph + Retrieval-Augmented Generation (KG-RAG)** architecture. We model the movie domain as a Neo4j graph, convert its facts into a corpus of natural-language triplets, index them with FAISS for sub-second semantic retrieval, and let a large language model produce grounded answers from the retrieved facts.

The target users are viewers asking conversational queries and researchers/engineers who need an explainable, fact-anchored alternative to opaque embedding-only retrievers.

Concrete contributions of this work:

- A reproducible Neo4j $\rightarrow$ 229,894-triplet $\rightarrow$ FAISS $\rightarrow$ RAG pipeline that answers natural-language movie queries with fact-grounded citations.
- A Streamlit-based interactive UI exposing the retrieved triplets, answer latency, lexical grounding score, and a per-query pipeline trace.
- An extended evaluation study on 51 queries across 3 Groq LLMs, with 15 Cypher-derived ground-truth labels and explicit handling of free-tier rate-limit, context-window, and payload constraints as evaluation evidence.
- An evaluation-driven model-selection rationale with `llama-3.1-8b-instant` as the production pick.

3. Background and Related Work

Movie recommendation has historically relied on collaborative and content-based signals, which can be opaque and brittle when users ask compositional questions such as constraints over actors, genres, years, and ratings. Knowledge graphs make explicit relationships between entities interpretable, while Retrieval-Augmented Generation (RAG) conditions language generation on retrieved evidence rather than model parameters alone. This section summarises prior work that motivates our design.

Lewis et al. [1] introduced Retrieval-Augmented Generation (RAG), which couples a dense passage retriever with a generative LLM so that responses are conditioned on retrieved evidence.

Hogan et al. [2] surveyed knowledge graphs as a structured way to represent inter-entity relationships, motivating their use as a retrieval substrate.

KG-Retriever [3] proposed efficient knowledge indexing for RAG over knowledge graphs, demonstrating gains from structured retrieval.

Nair and Cheriyan [4] applied particle filtering over multi-featured movie knowledge graphs to refine recommendations, demonstrating that graph structure can expose preferences that are hard to recover from interaction matrices alone.

Xu et al. [5] showed that KG-grounded RAG meaningfully improves factuality and answer specificity in customer-service question answering.

Our system synthesises these threads: we follow [1] for the RAG pattern, [2] for graph-structured knowledge representation, [3, 5] for indexing graph facts for retrieval-augmented answering, and [4] for structured movie-domain knowledge graphs as a practical recommendation substrate.

The key engineering choice is to **linearise** graph edges into natural-language sentences

before embedding, which lets us reuse off-the-shelf sentence encoders and a flat FAISS index while keeping the structural facts intact for the LLM to consume.

4. Dataset

We use the public Neo4j *recommendations* dataset hosted at <bolt://demo.neo4jlabs.com:7687>.

The dataset contains **28,865 nodes** and **166,262 relationships**.

4.1 Node Types

- **Movie** – title, year, IMDb rating, budget, revenue, runtime, language.
- **Actor / Director** – name, role, birth/death year.
- **User** – userId and rating history.
- **Genre** – genre label used to classify movies.

4.2 Relationship Types

- `(:Movie)-[:IN_GENRE]->(:Genre)`
- `(:User)-[:RATED]->(:Movie)`
- `(:Actor)-[:ACTED_IN]->(:Movie)`
- `(:Director)-[:DIRECTED]->(:Movie)`
- `(:Movie)-[:RELEASED]->(:Year)`

4.3 Dataset Visualizations

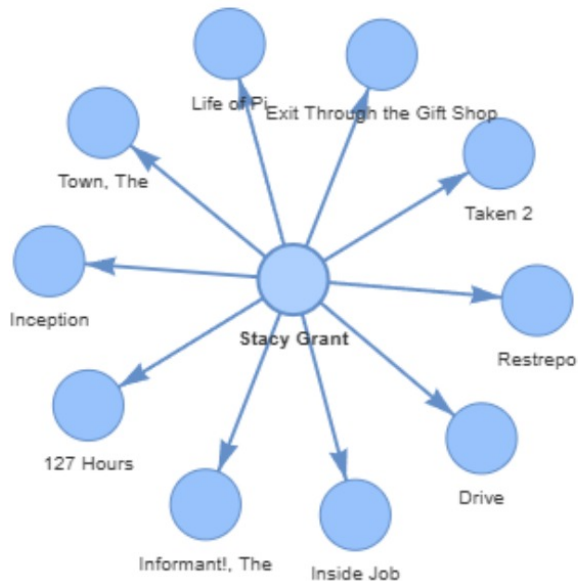


Figure 1: Here is a sample of the knowledge graph that has the RATED relationship. This shows the user named ‘Stacy Grant’ and what movies they gave a rating too.

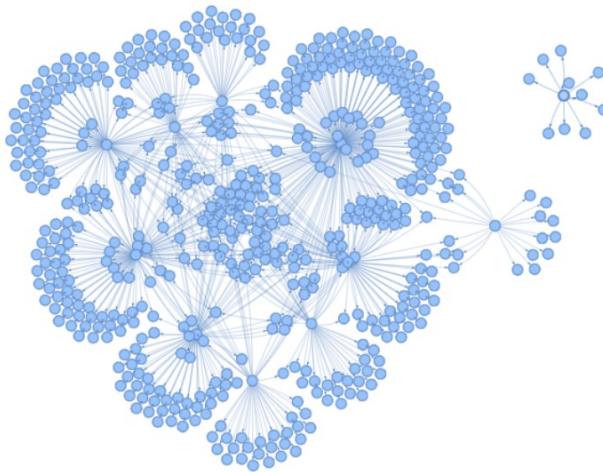


Figure 2: The following image displays the first 800 nodes in the same knowledge graph. The reason for choosing only 800 values is because if we tried to display all nodes the process would crash the visualization.

The graph provides a multi-relational structure connecting movies to genres, actors, directors, and users, which is suitable for multi-hop reasoning.

We collect single-hop facts in advance, render them as natural-language triplets, and let the LLM perform implicit multi-hop reasoning over the retrieved set.

5. System / Model / Algorithm Design

The pipeline has six logical stages:

1. **Knowledge Graph Construction:** Connect to Neo4j and execute Cypher queries to enumerate Movie/Actor/Director/Genre/User edges.
2. **Readable Triplet Generation:** Convert each edge into a subject-predicate-object tuple and then into a sentence, such as *“Toy Story was released in year 1995”*.
3. **Embedding Generation:** Encode each sentence with SentenceTransformers (all-MiniLM-L6-v2) into a 384-dimensional dense vector.
4. **FAISS Index Construction:** Store all 229,894 embeddings in a flat L2 index.
5. **Retrieval with Re-ranking:** Retrieve top- k FAISS neighbours and re-rank by cosine similarity.
6. **RAG Chain:** Feed top- k re-ranked triplets and the question into a LangChain RetrievalQA chain backed by a Groq-hosted LLM.

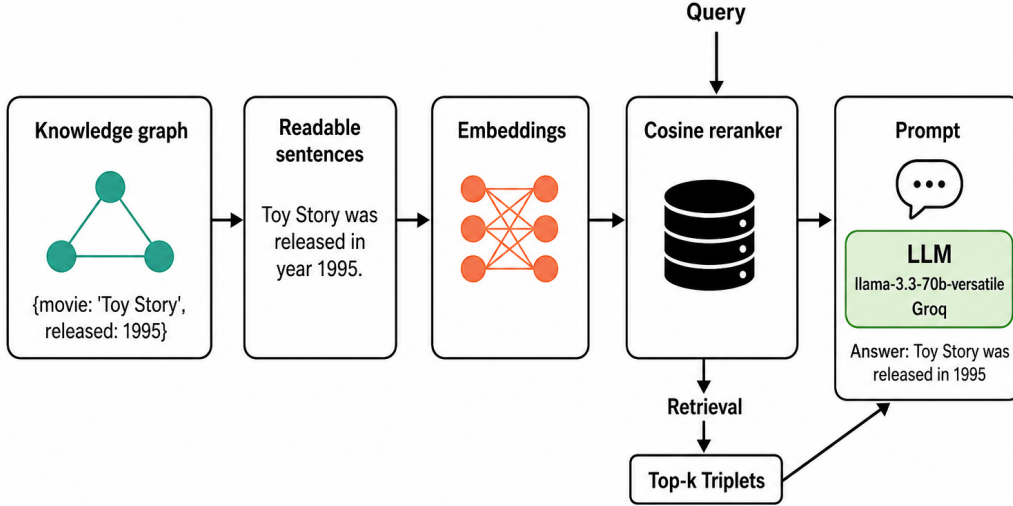


Figure 3: End-to-end KG-RAG pipeline. Knowledge graph triplets are linearised, embedded, indexed in FAISS, retrieved, re-ranked, and passed to a Groq-hosted LLM for answer generation.

6. Implementation Details

6.1 Languages and Frameworks

- Python 3.11 for pipeline and evaluation.
- Cypher for Neo4j queries.
- LangChain and langchain-groq for RetrievalQA orchestration.
- SentenceTransformers with all-MiniLM-L6-v2 embeddings.
- FAISS-CPU with IndexFlatL2 vector store.
- Streamlit for the interactive demo UI.
- Matplotlib and seaborn for plots.

6.2 Compute Environment

- Development: Google Colab CPU/T4 runtime.
- Embedding generation: CPU one-shot, approximately 10 minutes.
- Cached files: `triplet_embeddings.pkl` and `triplet_sentences.pkl`.
- Inference: Groq cloud LLM endpoints.
- Local FAISS query latency: under 50 ms.

6.3 Key Implementation Decisions

- Per-model retrieval k values were selected to respect context-window and payload limits.
- Robust answer parsing extracts a clean movie-title list across all models for consistent scoring.
- Exponential back-off and wall-clock budgets handle free-tier rate limits deterministically.
- Cypher-derived ground truth for 15 evaluation queries supports objective Precision, Recall, and F1.
- No credentials are stored in code; `GROQ_API_KEY` is read from the environment.

6.4 Pipeline Implementation Steps

A. Knowledge Graph Construction

The system first connects to the public Neo4j *recommendations* graph.

```
1 from langchain_community.graphs import Neo4jGraph
2
3 kg = Neo4jGraph(
4     url="bolt://demo.neo4jlabs.com:7687",
5     username="recommendations",
6     password="recommendations",
7     database="recommendations"
8 )
```

Cypher queries were used to extract movie properties, actor/director relationships, genres, and user ratings from the Neo4j graph.

B. Creating Readable Triplets

Extracted graph facts were converted into subject-predicate-object triplets and then rewritten as readable natural-language sentences.

```
1 triplets.append(("Toy Story", "was released in year", "1995"))
2
3 sentence = f"{subj} {rel} {obj}"
```

A total of **229,894 triplets** were created and converted into natural-language sentences.

C. Embedding Generation

Each triplet sentence was embedded using SentenceTransformers.

```
1 from sentence_transformers import SentenceTransformer
2
3 encoder_model = SentenceTransformer("all-MiniLM-L6-v2")
4
5 triplet_embeddings = encoder_model.encode(
6     triplet_sentences,
7     batch_size=64,
8     show_progress_bar=True
9 )
```

This generated **384-dimensional embeddings** for each triplet sentence.

D. FAISS Index Construction

The dense embeddings were indexed using FAISS for fast vector similarity search.

```

1 import faiss
2 import numpy as np
3
4 index = faiss.IndexFlatL2(384)
5 index.add(np.array(triplet_embeddings).astype("float32"))

```

All triplet embeddings were stored in the FAISS index to support efficient retrieval.

E. Retrieval with Re-ranking

The system retrieves candidate triplets and then re-ranks them using semantic cosine similarity.

```

1 from sentence_transformers import util
2
3 similarities = util.cos_sim(
4     query_embedding,
5     triplet_embeddings
6 )[0]
7
8 top_k_idx = similarities.argsort(descending=True)[:k]

```

The retrieval workflow uses FAISS with L2 distance for initial candidate search and cosine similarity for semantic re-ranking.

F. RAG Chain Creation

Finally, retrieved triplets are passed into a LangChain RetrievalQA chain backed by a Groq-hosted LLaMA model.

```

1 from langchain_groq import ChatGroq
2 from langchain.chains import RetrievalQA
3
4 qa_chain = RetrievalQA.from_chain_type(
5     llm=ChatGroq(
6         model_name="llama3.3-70b-versatile",
7         temperature=0
8     ),
9     retriever=vectorstore.as_retriever(),
10    chain_type="stuff",
11    chain_type_kwargs={"prompt": prompt_template}
12 )

```

The RetrievalQA chain generates fact-grounded movie recommendations using the retrieved knowledge-graph triplets as supporting context.

Together, these steps create a recommendation engine that is accurate, scalable, and interpretable. The implementation choices support efficient retrieval, semantic understanding of user queries, and grounded answer generation.

Google Colab:

<https://github.com/abharathkumarr/Graph-Based-Intelligent-Movie-Recommendation-System-Using-RAG>

7. Task Distribution and Contributions

- **Knowledge Graph Connection and Data Extraction:** Main contributor: Venkata Siva Sai Krishna Prasad Yedupati. Connected the system to the Neo4j recommendations dataset, wrote Cypher queries, extracted graph facts, and validated extracted nodes and relationships.
- **Triplet Generation and Preprocessing:** Main contributor: Venkata Siva Sai Krishna Prasad Yedupati. Converted extracted graph relationships into subject-predicate-object triplets and prepared natural-language triplet sentences for embedding.
- **Embedding Generation and FAISS Retrieval:** Main contributor: Bharath Kumar A. Generated sentence embeddings using SentenceTransformers, built the FAISS index, and implemented semantic retrieval and re-ranking.
- **RAG Pipeline and LLM Integration:** Main contributor: Bharath Kumar A. Designed the LangChain RetrievalQA pipeline, integrated Groq-hosted LLMs, and connected retrieved graph facts to answer generation.
- **Evaluation and Testing:** Main contributors: Bharath Kumar A and Venkata Siva Sai Krishna Prasad Yedupati. Created evaluation queries, compared RAG outputs with Cypher-derived ground truth, calculated Precision@10, Recall@10, F1@10, and tested system correctness.
- **User Interface and Demo:** Main contributor: Sriya Verma Saripella. Prepared the Streamlit demo interface, organized screenshots, validated demo outputs, and supported presentation preparation.
- **System Integration and Final Validation:** Main contributors: All team members. Integrated the Neo4j extraction, triplet generation, FAISS retrieval, RAG pipeline, evaluation workflow, and Streamlit UI into one end-to-end system. All members participated in debugging, final testing, and demo validation.
- **Report and Presentation:** Main contributors: All team members. All members contributed to final report writing, slide preparation, integration review, and final

presentation.

8. Evaluation and Testing Results

Evaluation is the most critical section of this report. We present two complementary studies: a focused qualitative validation on 2 queries, and an extended 51-query x 3-model quantitative study that drives model selection.

8.1 Metrics

- **Precision@10** – proportion of top-10 returned movies matching Cypher-derived ground truth.
- **Recall@10** – proportion of all ground-truth movies appearing in the top-10.
- **F1@10** – harmonic mean of Precision@10 and Recall@10.
- **Latency** – end-to-end wall-clock time per query.
- **Non-empty answer rate** – fraction of queries returning any text.

8.2 Focused Qualitative Validation

Query A: *“Find movies acted by Leonardo DiCaprio released after 2000.”*

- Cypher ground truth includes titles such as Gangs of New York, Catch Me If You Can, The Aviator, Blood Diamond, The Departed, Revolutionary Road, Inception, Django Unchained, The Wolf of Wall Street, and The Revenant.
- RAG output returned 10 correct titles out of 14.
- Precision@10 = 0.70, Recall@10 = 0.50, F1@10 = 0.58.

Query B: *“Recent Adventure movie rated above 3 with Leonardo DiCaprio.”*

- Cypher ground truth: The Revenant, Blood Diamond, The Beach.
- RAG output: The Revenant (2015), correctly identified with explanation.

8.3 Extended Evaluation: 51 Queries x 3 Models

We evaluated three Groq-hosted LLMs: llama-3.3-70b-versatile, llama-3.1-8b-instant, and openai/gpt-oss-20b.

Fifteen queries have Cypher-derived ground truth used for Precision, Recall, and F1. The remaining 36 queries contribute latency and non-empty-answer-rate evidence.

Table 1: Per-model averages across 51 queries.

Model	P@10	R@10	F1@10	Latency (s)
llama-3.1-8b-instant	0.707	0.225	0.317	2.97
openai/gpt-oss-20b	0.507	0.165	0.225	6.59
llama-3.3-70b-versatile	0.127	0.019	0.031	7.92

8.4 Model Quality and Latency Comparison

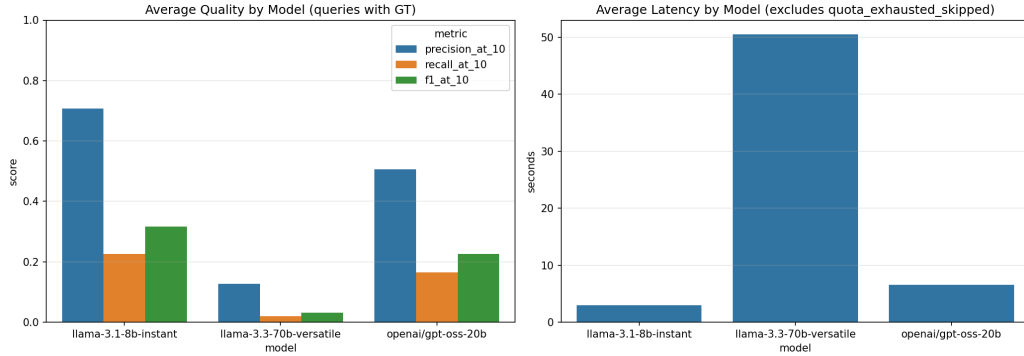


Figure 4: Average quality and latency comparison across evaluated models. llama-3.1-8b-instant provides the best balance between F1 score, latency, and robustness.

8.5 Precision and Recall vs. Retrieval k

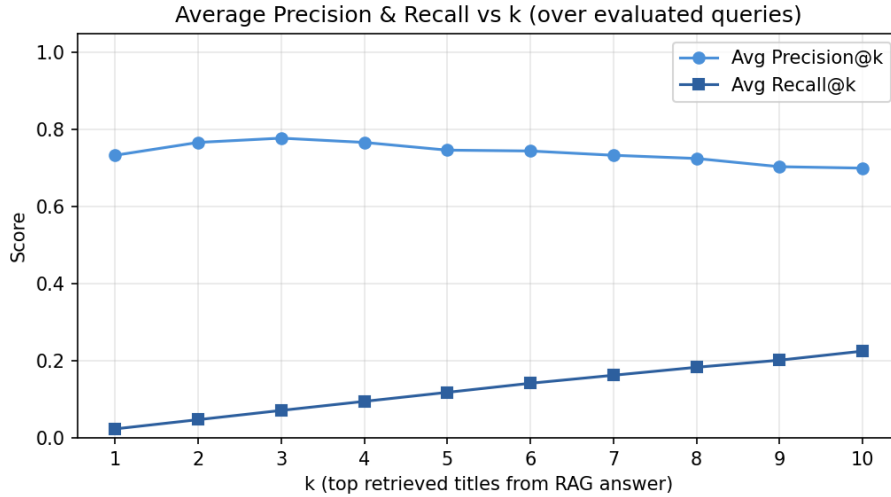


Figure 5: Average Precision@k and Recall@k for the production model. Precision remains stable while recall increases as k grows.

8.6 Per-Query Retrieval Metrics

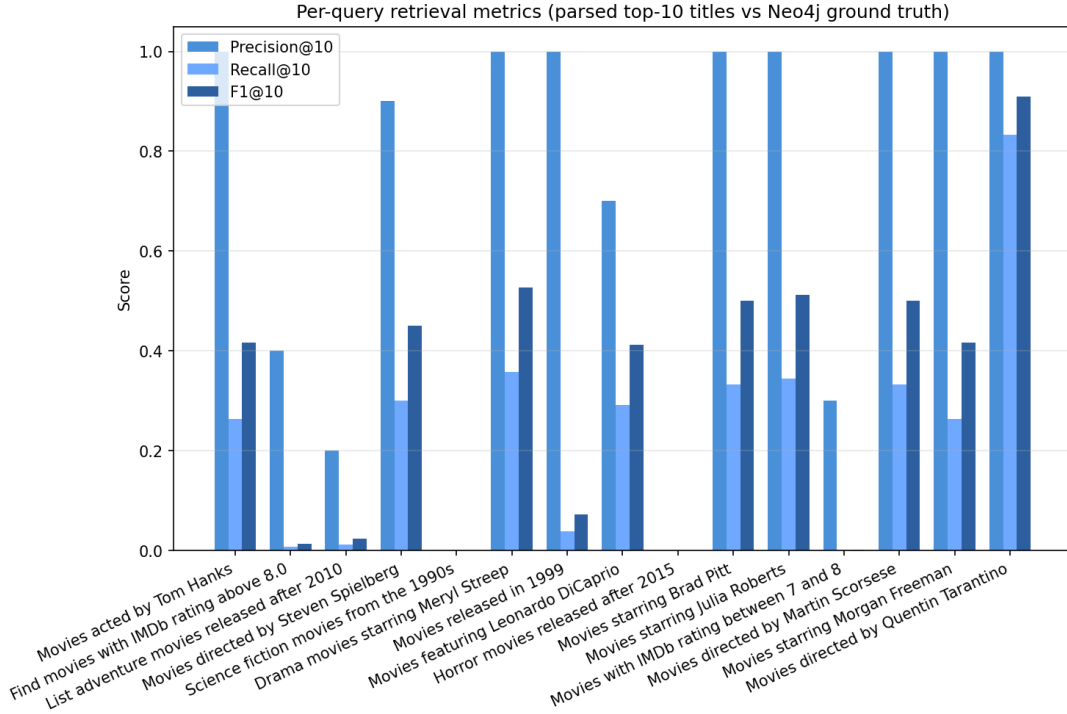


Figure 6: Per-query Precision@10, Recall@10, and F1@10 for the production model across the 15 ground-truth queries.

8.7 Retrieval Confusion Matrix

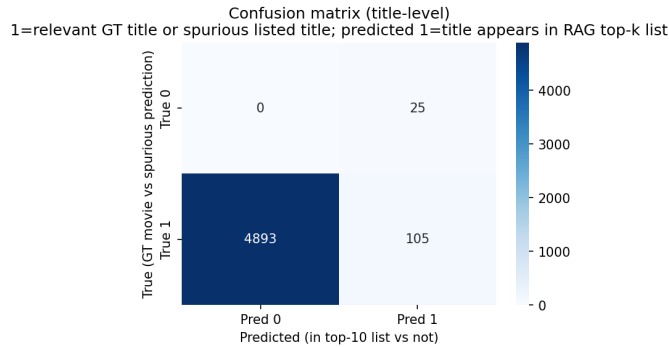


Figure 7: Title-level confusion matrix for llama-3.1-8b-instant. The model produced 105 true positives, 4893 false negatives, and 25 false positives. False negatives dominate because many queries have large ground-truth sets while the model returns only top-10 results.

8.8 Findings

Production pick: llama-3.1-8b-instant is the recommended production model. It achieves the highest F1@10 and the lowest mean latency while returning non-empty answers for all 51 queries.

Deployment finding: Pure offline accuracy comparisons can be misleading because provider-side rate limits, context-window caps, and payload-size caps materially affect real-world deployment.

Recall limitation: Recall@10 is structurally limited because many queries have ground-truth sets of 50–200 titles, while the LLM is prompted to return at most 10 titles.

8.9 Reproduction Steps

```
1 REPO=Graph-Based-Intelligent-Movie-Recommendation-System-Using-RAG
2
3 git clone https://github.com/abharathkumarr/$REPO.git
4 cd $REPO
5
6 pip install -r requirements.txt
7 export GROQ_API_KEY="<your_key>"
8
9 jupyter notebook CMPE258_Project_Code.ipynb
10
11 # Run all cells under Section "Option 2"
12 # to regenerate evaluation artefacts
13
14 ls evaluation/eval_detailed_results.csv
15 ls evaluation/eval_model_summary.csv
```

Listing 1: Commands to reproduce evaluation results

Correctness checks:

- Ground truth for each evaluation query is generated by a deterministic Cypher query against the same Neo4j graph used by the RAG retriever.
- Partial evaluation CSV files are written after each model finishes.
- The Streamlit UI exposes retrieved triplets, prompt, raw answer, and a lexical grounding score for qualitative verification.

9. User Interface and Demo

The Streamlit demo accepts natural-language queries, displays the answer, grounding score, retrieved triplets, and pipeline trace. Model switching, latency, and pipeline details are exposed in the sidebar.

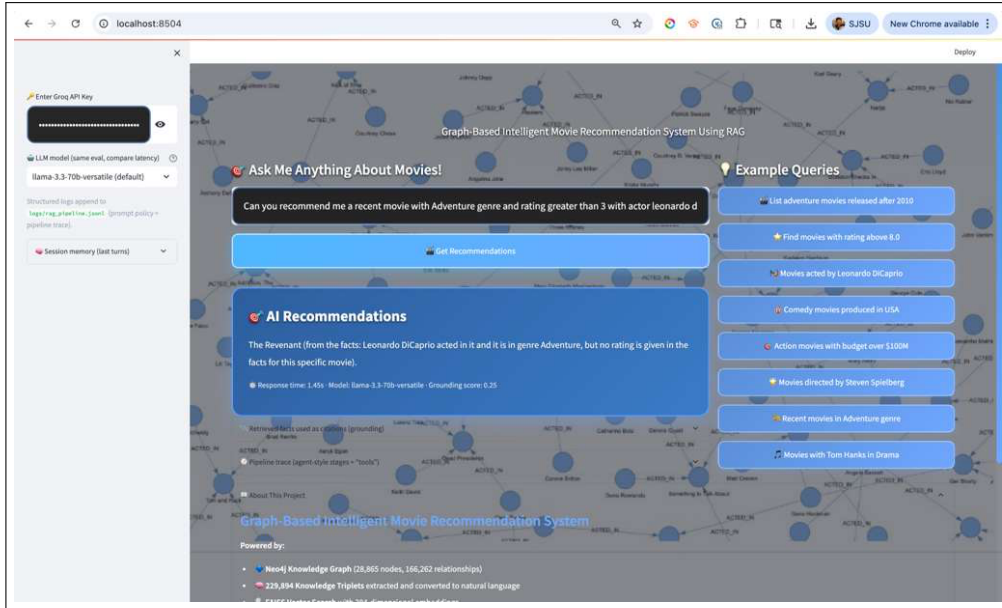


Figure 8: Live demo of the Streamlit UI answering a movie recommendation query using retrieved knowledge-graph facts.

10. Conclusion and Future Work

We presented an evaluation-first Knowledge-Graph + RAG movie recommendation system that converts 229,894 Neo4j edges into a FAISS-indexed natural-language corpus, retrieves relevant facts for each query, and generates grounded answers via a Groq-hosted LLM.

The 51-query x 3-model evaluation backs an explicit production choice: `llama-3.1-8b-instant`. The study also shows that free-tier rate limits, context-window limits, and payload-size constraints should be treated as first-class deployment evidence.

Future work:

- Hybrid retrieval combining dense FAISS retrieval with on-demand Neo4j Cypher traversal.
- Personalisation using per-user rating history as additional triplets.
- A learned cross-encoder re-ranker to improve retrieval quality.
- Expansion to larger movie graphs such as IMDb or TMDB.
- Multi-modal upgrades incorporating posters and trailers.

References

- [1] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *arXiv:2005.11401*, 2020. <https://arxiv.org/abs/2005.11401>
- [2] A. Hogan et al., “Knowledge Graphs,” *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–37, 2022. <https://doi.org/10.1145/3447772>
- [3] “KG-Retriever: Efficient Knowledge Indexing for Retrieval-Augmented Large Language Models,” *arXiv:2412.05547*, 2023. <https://arxiv.org/html/2412.05547v1>
- [4] L. S. Nair and J. Cheriyan, “Multi-Featured Movie Recommendation Using Knowledge Graph,” *IDCIoT 2023*. <https://doi.org/10.1109/idciot56793.2023.10053435>
- [5] Z. Xu et al., “Retrieval-Augmented Generation with Knowledge Graphs for Customer Service Question Answering,” *SIGIR 2024*. <https://doi.org/10.1145/3626772.3661370>
- [6] Neo4j, “Example datasets – Getting Started.” <https://neo4j.com/docs/getting-started/appendix/example-data/>
- [7] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” *EMNLP 2019*. <https://arxiv.org/abs/1908.10084>
- [8] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, 2019. <https://arxiv.org/abs/1702.08734>
- [9] LangChain, “RetrievalQA Documentation.” https://python.langchain.com/docs/modules/chains/popular/vector_db_qa
- [10] Groq, “LLM Inference API Documentation.” <https://console.groq.com/docs>